

Documento Técnico Exhaustivo:

Desarrollo de ScrapSAE Extension & Web

Proyecto: ScrapSAE.Extension y ScrapSAE.Web **Rama:** extension1 **Fecha:** Marzo 2026

Este documento técnico detalla la arquitectura, los componentes, los modelos de datos, los endpoints y los flujos de trabajo necesarios para construir la extensión de Chrome y la plataforma web de ScrapSAE. Sirve como guía definitiva para el equipo de desarrollo.

1. Arquitectura del Sistema

El sistema evoluciona de una aplicación de escritorio monolítica a una arquitectura distribuida y orientada a servicios (SaaS).

1.1. Componentes Principales

Componente	Tecnología	Responsabilidad Principal
ScrapSAE.Extension	Vite, TypeScript, HTML/CSS	Frontend inyectado en el navegador. Extrae datos, simula comportamiento humano y genera Excel.
ScrapSAE.Web	HTML, CSS, JS Vanilla	Landing page comercial, gestión de suscripciones y portal de usuario.
ScrapSAE.Api	ASP.NET Core (C#)	Backend central. Procesa datos con IA, gestiona webhooks de Stripe y sirve a la extensión.
Supabase	PostgreSQL, Auth, Storage	Base de datos centralizada, autenticación de usuarios y almacenamiento de layouts.
Stripe	API de Pagos	Pasarela de pagos, gestión de suscripciones y facturación.

1.2. Flujo de Datos (End-to-End)

1. El usuario inicia sesión en la extensión mediante Supabase Auth.
 2. El usuario navega a la página del proveedor (ej. Festo) y abre el Popup de la extensión.
 3. Selecciona un layout y hace clic en “Iniciar Scraping”.
 4. El **Content Script** se inyecta, aplica técnicas de evasión (`stealth_script.js`), simula movimientos de ratón y extrae el HTML/JSON basado en los selectores.
 5. Los datos crudos se envían al **Background Service Worker**.
 6. El Service Worker envía los datos a `ScrapSAE.Api` (`/api/scraping/process`).
 7. El API utiliza OpenAI para limpiar y estructurar los datos, devolviendo un JSON estructurado.
 8. El Service Worker recibe el JSON, utiliza `SheetJS` para generar un archivo `.xlsx` según el layout del usuario y lo descarga mediante `chrome.downloads` .
 9. Se registra la ejecución en Supabase (`execution_reports`).
-

2. Modelos de Datos (TypeScript)

Para mantener la coherencia con el backend en C#, se deben replicar los siguientes DTOs en TypeScript dentro del proyecto `ScrapSAE.Extension/src/shared/types.ts` .

2.1. Modelos de Scraping

```
export interface ScrapedProduct {
  skuSource?: string;
  title?: string;
  description?: string;
  rawHtml?: string;
  screenshotBase64?: string;
  imageUrl?: string;
  imageUrls: string[];
  price?: number;
  category?: string;
  brand?: string;
  sourceUrl?: string;
  attributes: Record<string, string>;
  navigationUrls: string[];
  scrapedAt: string; // ISO Date
  aiEnriched: boolean;
  attachments: ProductAttachment[];
}

export interface ProductAttachment {
  fileName: string;
  fileUrl: string;
  fileType?: string;
  fileSizeBytes?: number;
}

export interface ProcessedProduct {
  sku?: string;
  name: string;
  brand?: string;
  model?: string;
  description: string;
  features: string[];
  specifications: Record<string, string>;
  suggestedCategory?: string;
  categories: string[];
  lineCode?: string;
  price?: number;
  currency?: string;
  stock?: number;
  images: string[];
  attachments: ProductAttachment[];
  confidenceScore?: number;
}
```

```
originalRawData?: string;  
}
```

2.2. Modelo de Selectores (Layout)

Este modelo es crítico, ya que define cómo el Content Script leerá el DOM.

```
export interface SiteSelectors {  
  productListSelector?: string;  
  productListClassPrefix?: string;  
  productCardClassPrefix?: string;  
  productLinkSelector?: string;  
  categoryLandingUrl?: string;  
  categoryLinkSelector?: string;  
  categoryNameSelector?: string;  
  categorySearchTerms: string[];  
  searchInputSelector?: string;  
  searchButtonSelector?: string;  
  titleSelector?: string;  
  priceSelector?: string;  
  descriptionSelector?: string;  
  imageSelector?: string;  
  skuSelector?: string;  
  categorySelector?: string;  
  brandSelector?: string;  
  nextPageSelector?: string;  
  detailButtonText?: string;  
  detailButtonClassPrefix?: string;  
  variantTableSelector?: string;  
  variantRowSelector?: string;  
  variantSkuLinkSelector?: string;  
  detailSkuSelector?: string;  
  detailPriceSelector?: string;  
  usesInfiniteScroll: boolean;  
  maxPages: number;  
  scrapingMode?: string; // "traditional" o "families"  
  productFamilyLinkSelector?: string;  
  productFamilyLinkText?: string;  
  categoryUrls?: string[];  
}
```

3. Desarrollo de la Extensión (`ScrapSAE.Extension`)

3.1. Estructura del Proyecto

```
ScrapSAE.Extension/  
├─ public/  
│   ├── manifest.json  
│   ├── stealth_script.js  
│   └─ icons/  
├─ src/  
│   ├── background/  
│   │   └─ service_worker.ts      # Orquestador, SheetJS, llamadas API  
│   ├── content/  
│   │   ├── extractor.ts          # Lógica de lectura del DOM  
│   │   └─ human_simulator.ts     # Movimientos de ratón, pausas  
│   ├── popup/  
│   │   ├── popup.html  
│   │   ├── popup.css  
│   │   └─ popup.ts               # UI principal (Iniciar, Progreso)  
│   ├── sidepanel/  
│   │   ├── sidepanel.html  
│   │   ├── sidepanel.css  
│   │   └─ sidepanel.ts           # Gestión de Layouts e Historial  
│   └─ shared/  
│       ├── types.ts              # Interfaces DTO  
│       └─ supabase_client.ts     # Cliente de Supabase Auth/DB  
├─ package.json  
└─ vite.config.ts
```

3.2. Módulo de Simulación Humana (`human_simulator.ts`)

Para evitar la detección de bots, el Content Script debe implementar pausas y movimientos realistas.

- **Pausas Aleatorias:** Implementar una función `delay(min, max)` que garantice una pausa mínima de 1 segundo (1000ms) entre interacciones, con variación aleatoria para no ser predecible.
- **Movimiento de Ratón:** Utilizar curvas de Bézier para simular el trayecto del cursor desde su posición actual hasta el elemento objetivo antes de hacer clic.

- **Scroll Progresivo:** No saltar directamente al final de la página. Hacer scroll en incrementos variables con pequeñas pausas, simulando la lectura humana.

3.3. Generación de Excel (`service_worker.ts`)

El Service Worker utilizará la biblioteca `xlsx` (SheetJS) para crear el archivo.

1. Recibe un array de `ProcessedProduct` .
 2. Mapea las propiedades del objeto a las columnas definidas en el layout del usuario.
 3. Crea un libro de trabajo (`XLSX.utils.book_new()`) y una hoja (`XLSX.utils.json_to_sheet()`).
 4. Genera el archivo en formato Base64.
 5. Utiliza `chrome.downloads.download({ url: "data:application/vnd.openxmlformats-officedocument.spreadsheetml.sheet;base64," + base64Data, filename: "ScrapSAE_Export.xlsx" })` .
-

4. Desarrollo del Backend (`ScrapSAE.Api`)

El backend existente debe ampliarse para soportar la extensión y la monetización.

4.1. Nuevos Endpoints Requeridos

Método	Endpoint	Descripción
POST	/api/extension/process	Recibe <code>ScrapedProduct[]</code> , lo pasa por <code>IAIProcessorService</code> y devuelve <code>ProcessedProduct[]</code> .
GET	/api/layouts	Obtiene los layouts guardados del usuario autenticado (vía token JWT de Supabase).
POST	/api/layouts	Crea o actualiza un layout personalizado.
POST	/api/stripe/create-checkout	Genera una sesión de Stripe Checkout para el plan seleccionado.
POST	/api/stripe/webhook	Recibe eventos de Stripe (<code>checkout.session.completed</code>) y actualiza Supabase.

4.2. Modificaciones en Supabase

Se deben ejecutar los siguientes scripts SQL para preparar la base de datos:

```

-- Tabla para perfiles de usuario y suscripciones
CREATE TABLE IF NOT EXISTS user_profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email VARCHAR(255) NOT NULL,
  stripe_customer_id VARCHAR(100),
  subscription_status VARCHAR(50) DEFAULT 'free', -- free, pro, enterprise
  plan_type VARCHAR(50) DEFAULT 'free',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Tabla para layouts personalizados
CREATE TABLE IF NOT EXISTS user_layouts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  name VARCHAR(100) NOT NULL,
  selectors JSONB NOT NULL DEFAULT '{}',
  column_mapping JSONB NOT NULL DEFAULT '{}', -- Mapeo de ProcessedProduct
a columnas Excel
  is_default BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Actualizar execution_reports
ALTER TABLE execution_reports ADD COLUMN IF NOT EXISTS user_id UUID
REFERENCES auth.users(id);

-- Habilitar RLS
ALTER TABLE user_layouts ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can manage their own layouts" ON user_layouts FOR ALL
USING (auth.uid() = user_id);

```

5. Desarrollo de la Plataforma Web (ScrapSAE .web)

La Landing Page es el motor de ventas. Se construirá con HTML/CSS/JS Vanilla para máxima velocidad de carga y SEO.

5.1. Estructura de la Ficha Técnica (Páginas)

- **Inicio (`index.html`):**
 - **Hero:** Título claro (“Extrae datos a Excel mágicamente”), subtítulo, botón CTA principal (“Instalar Extensión - Es Gratis”).
 - **Video Demo:** Animación o video corto mostrando el flujo: Clic en extensión -> Scraping -> Descarga de Excel.
 - **Beneficios (Neuromarketing):** Enfoque en ahorro de tiempo, eliminación de errores manuales y evasión de bloqueos (simulación humana).
 - **Pricing:** Tabla de 3 columnas (Free, Pro, Enterprise) con botones de suscripción que llaman al endpoint `/api/stripe/create-checkout`.
- **Política de Privacidad (`privacy.html`):** Documento legal obligatorio detallando que no se venden datos y qué información se almacena (email, layouts).
- **Términos de Servicio (`terms.html`):** Condiciones de uso de la herramienta SaaS.

5.2. Integración Stripe (Frontend)

Los botones de “Actualizar a Pro” en la web y en la extensión redirigirán al usuario a una URL generada por el backend:

```
// Ejemplo de llamada desde la web
async function subscribeToPro() {
  const response = await
  fetch('https://api.scrapxae.com/api/stripe/create-checkout', {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${supabaseToken}` },
    body: JSON.stringify({ plan: 'pro' })
  });
  const data = await response.json();
  window.location.href = data.checkoutUrl; // Redirige a Stripe
}
```

6. Automatización y Despliegue (CI/CD)

Se utilizará GitHub Actions para automatizar el ciclo de vida del software.

6.1. Flujo de Backend (`deploy-api.yml`)

Se activa al hacer push a `main` si hay cambios en `src/ScrapSAE.Api/`. Compila el proyecto .NET, ejecuta pruebas y despliega en el servidor de producción.

6.2. Flujo de Web (`deploy-web.yml`)

Se activa al hacer push a `main` si hay cambios en `ScrapSAE.Web/`. Sincroniza los archivos estáticos HTML/CSS/JS con el hosting (ej. Cloudflare Pages o AWS S3).

6.3. Flujo de Extensión (`build-extension.yml`)

Se activa al crear un nuevo *Tag* en Git (ej. `v1.0.0`).

1. Instala dependencias (`npm install`).
2. Compila el proyecto (`npm run build`).
3. Empaqueta la carpeta `dist/` en un archivo `ScrapSAE_Extension_v1.0.0.zip` .
4. Sube el `.zip` como un *Release Asset* en GitHub, listo para ser descargado y subido manualmente a la Chrome Web Store.

7. Requisitos de Publicación

Para asegurar la aprobación en las tiendas, se deben cumplir estrictamente los siguientes puntos:

7.1. Chrome Web Store

- **Single Purpose:** La extensión debe tener un único propósito claro: “Extraer datos de productos de páginas web y exportarlos a Excel”.
- **Permisos Justificados:**

- `activeTab` : Para leer el DOM de la página actual.
- `storage` : Para guardar el token de sesión y la configuración local.
- `downloads` : Para guardar el archivo `.xlsx` generado.
- `scripting` : Para inyectar el script de simulación humana.
- **Assets:** Icono de 128x128px, al menos 1 captura de pantalla de 1280x800px, y una imagen promocional de 440x280px.

7.2. Firefox Add-ons (AMO)

- Asegurar que el `manifest.json` incluya la clave `browser_specific_settings.gecko.id` con un ID único (ej. `extension@scrapsae.com`).
- El código no debe usar ofuscación severa (minificación estándar de Vite es aceptable).